

Análisis y Diseño

Segunda Parte: Orientado a Objetos	2
Enfoque	2
1.1. Análisis	2
1.2. Diseño	2
1.3. Programación	2
Definiciones del Paradigma	2
2.1. Objetos	2
2.2. Clases	3
2.3. Atributos	3
2.4. Métodos	3
2.5. Herencia	3
UML	4
3.1. Cosas	4
3.2. Relaciones	4
3.3. Diagramas	4
3.3.1. Diagrama de Casos de Uso	5
3.3.2. Diagrama de Actividad	8
3.3.3. Diagrama de Secuencia	10
3.3.4. Diagrama de Clases	12

Segunda Parte: Orientado a Objetos

1. Enfoque

El análisis y diseño a objetos (ADOO) es un enfoque en ingeniería de software que modela un sistema como un grupo de objetos que interactúan entre sí.

El proceso de diseño orientado a objetos comprende el diseño de clases de objetos y las relaciones entre estas clases. Las clases definen los objetos del sistema y sus interacciones. Cuando el diseño se implementa como un programa ejecutable, los objetos requeridos se crean dinámicamente utilizando las definiciones de las clases.

¡Divide y vencerás! El paradigma orientado a objetos se basa en dividir el “PROBLEMA” (para lo que construimos el software) en problemas más pequeños que funcionan como “mini programas”. Cada objeto es una representación de alguna cosa o evento real. Los objetos no se crean al azar, representan parte del problema a resolver. Pueden ser zapatos, compras, ventas, facturas, recibos, empleados, clientes...

La Orientación a objetos es algo que se conoce como PARADIGMA de programación. No es un lenguaje en sí mismo, sino un conjunto de lenguajes, de herramientas y procedimientos soportados por varios lenguajes de programación.

1.1. Análisis

- Comprende el desarrollo de un modelo orientado a objetos del dominio de aplicación.
- Los objetos identificados reflejan las entidades y operaciones que se asocian con el problema a resolver.
- En esta etapa se utiliza una combinación de texto y diagramas para comprender los requisitos del sistema.
- Se enfoca en la descripción de los datos, las funciones y los comportamientos REQUERIDOS.

1.2. Diseño

- Comprende el desarrollo de un modelo orientado a objetos de un sistema software para implementar los requerimientos identificados.
- Los objetos están relacionados con la solución del problema por resolver.
- Pueden existir relaciones estrechas entre algunos objetos del problema y de la solución, pero inevitablemente se tienen que agregar nuevos objetos para transformar el problema e implementar la solución.
- Proporciona detalles acerca de las estructuras de datos, las arquitecturas, las interfaces y los componentes de software necesarios para implementar el sistema.

1.3. Programación

- Se refiere a implementar el diseño de software utilizando un lenguaje de programación orientado a objetos, como Java.
- Un lenguaje orientado a objetos provee los recursos para definir las clases y un sistema para crear los objetos correspondientes a las clases.

2. Definiciones del Paradigma

2.1. Objetos

- Los objetos son personas, lugares o cosas relevantes para el sistema a analizar.
- Los sistemas orientados a objetos describen las entidades como objetos.
- Algunos objetos comunes son clientes, artículos, pedidos, etcétera.
- Los objetos también pueden ser pantallas de GUI o áreas de texto en la pantalla

2.2. Clases

- Los objetos se representan y agrupan mediante clases, las cuales son óptimas para la reutilización y la facilidad de mantenimiento.
- Una clase define el conjunto de atributos compartidos y comportamientos que se encuentran en cada objeto de la clase.
- El término instanciar se utiliza cuando se crea un objeto a partir de una clase. Ej.: un programa podría instanciar un estudiante llamado Peter Wellington como un objeto de la clase etiquetada como Estudiante
- Por lo general, los nombres de las clases son sustantivos o frases cortas y empiezan con mayúscula.

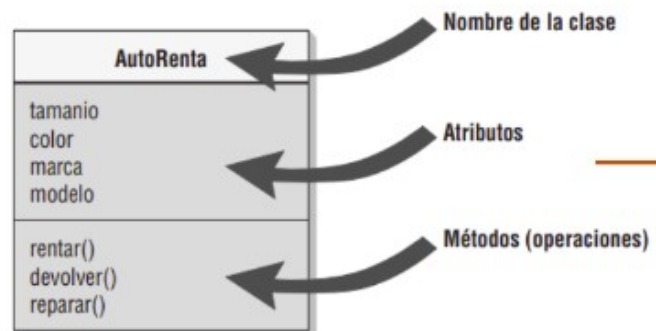
2.3. Atributos

Un atributo describe cierta propiedad que poseen todos los objetos de la clase.

En un auto, los atributos pueden ser: tamaño, color, marca y modelo. Todos los autos poseen estos atributos, pero cada automóvil tendrá distintos valores para sus atributos. Por ejemplo, un auto puede ser azul, blanco o de algún otro color.

2.4. Métodos

- Un método es una acción que se puede solicitar de cualquier objeto de la clase.
- Los métodos son los procesos que una clase sabe cómo llevar a cabo.
- También se les conoce como operaciones.
- Para la clase **Auto**, **rentar()**, **devolver()** y **reparar()** son ejemplos de métodos.
- Al especificar métodos, por lo general la primera letra está en minúscula.



2.5. Herencia

- Las clases pueden tener hijos; es decir, se puede crear una clase a partir de otra.
- En UML, la clase original (o padre) se conoce como clase base; a la clase hija se le denomina clase derivada. Podemos crear una clase derivada de tal forma que herede todos los atributos y comportamientos de la clase base. Sin embargo, una clase derivada puede tener atributos y comportamientos adicionales.
- La herencia reduce la labor de programación al permitir que se utilicen los objetos comunes con facilidad.



3. UML

En paralelo a este paradigma OO, veremos un lenguaje de modelado que es un estándar en la industria para modelar sistemas orientados a objetos “LENGUAJE DE MODELADO UNIFICADO” o UML por sus siglas en inglés. El conjunto de herramientas de UML incluye diagramas que permiten visualizar la construcción de un sistema orientado a objetos. Cada iteración aborda de manera cada vez más detallada el diseño del sistema, hasta que las cosas y las relaciones en el sistema estén definidas con claridad y precisión en documentos de UML. El UML es una potente herramienta que puede mejorar en forma considerable la calidad de nuestro análisis y diseño de sistemas, y en consecuencia puede ayudarnos a crear sistemas de información de mayor calidad.

3.1. Cosas

El UML consiste en cosas, relaciones y diagramas. Los primeros componentes (o elementos primarios de uso y muchos otros elementos que proveen la forma de crear modelos. Las cosas estructurales permiten al usuario describir las relaciones. Las cosas de comportamiento describen la forma en que funcionan las cosas. Algunos ejemplos de cosas de comportamiento son las interacciones y las máquinas de estado. Las cosas de grupo se utilizan para definir límites. El paquete es un ejemplo de cosa de grupo. Por último tenemos las cosas de anotaciones, para poder agregar notas en los diagramas.

Categoría de UML	Elementos de UML	Detalles específicos de UML
Cosas	Cosas estructurales	Clases Interfaces Colaboraciones Casos de uso Clases activas Componentes Nodos
	Cosas de comportamiento	Interacciones Máquinas de estado
	Cosas de agrupamiento	Paquetes
	Cosas de anotaciones	Notas

3.2. Relaciones

Las relaciones son el pegamento que mantiene las cosas unidas entre sí. Es conveniente pensar en las relaciones de dos formas.

1. Las relaciones estructurales se utilizan para unir las cosas en los diagramas estructurales. Las relaciones estructurales incluyen dependencias, agregaciones, asociaciones y generalizaciones. Por ejemplo, las relaciones estructurales muestran herencia.

Categoría de UML	Elementos de UML	Detalles específicos de UML
Relaciones	Relaciones estructurales	Dependencias Agregaciones Asociaciones Generalizaciones
	Relaciones de comportamiento	Comunica Incluye Extiende Generaliza

2. Las relaciones de comportamiento se utilizan en los diagramas de comportamiento. Los cuatro tipos básicos de relaciones de comportamiento son comunica, incluye, extiende y generaliza.

3.3. Diagramas

Hay dos tipos principales de diagramas en UML: diagramas estructurales y diagramas de comportamiento.

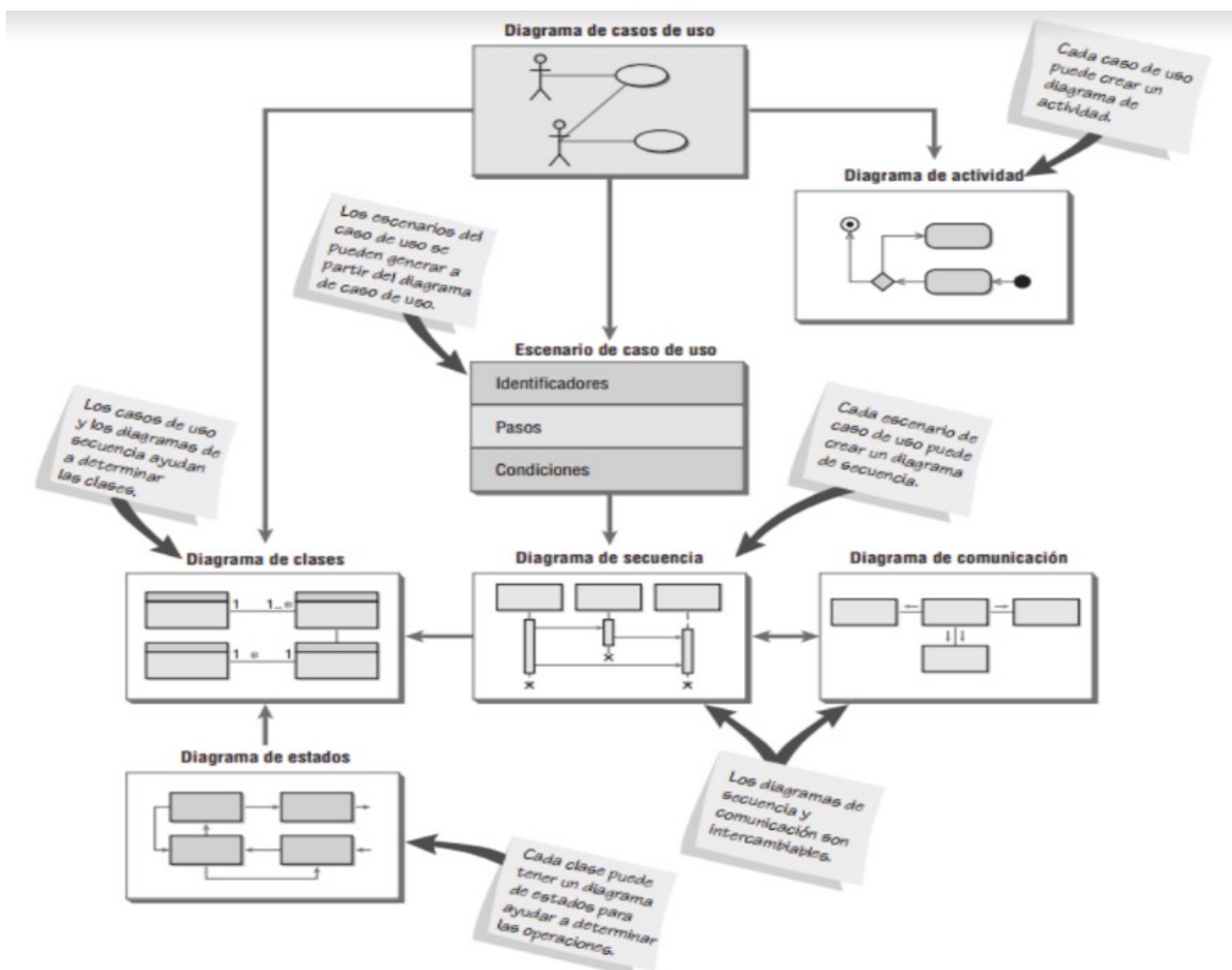
1. Los diagramas estructurales se utilizan, por ejemplo, para describir las relaciones entre las clases. Éstos se dividen en diagramas de clases, diagramas de objetos, diagramas de componentes y diagramas de despliegue. Por otro lado, los diagramas de comportamiento se pueden utilizar para describir la interacción entre las personas (actores en UML) y lo que denominamos caso de uso, o la forma en que los actores utilizan el sistema.

Categoría de UML	Elementos de UML	Detalles específicos de UML
Diagramas	Diagramas estructurales	Diagramas de clases Diagramas de componentes Diagramas de despliegue
	Diagramas de comportamiento	Diagramas de casos de uso Diagramas de secuencia Diagramas de comunicación Diagramas de estados Diagramas de actividad

2. Los diagramas de comportamiento se dividen en diagramas de casos de uso, diagramas de secuencia, diagramas de comunicación, diagramas de estados y diagramas de actividad

Los seis diagramas de UML que se utilizan con más frecuencia son:

- Un diagrama de casos de uso, que describe la forma en que se utiliza el sistema.
- Un escenario de caso de uso (aunque técnicamente no es un diagrama). Este escenario es una articulación verbal de excepciones para el comportamiento principal descrito por el caso de uso principal.
- Un diagrama de actividad, que ilustra el flujo de actividades en general. Cada caso de uso puede crear un diagrama de actividad.
- Los diagramas de secuencia, que muestran la secuencia de las actividades y las relaciones entre las clases. Cada caso de uso puede crear uno o más diagramas de secuencia. El diagrama de comunicación es la alternativa a un diagrama de secuencia, el cual contiene la misma información pero enfatiza la comunicación en vez de la sincronización.
- Los diagramas de clases, que muestran las clases y sus relaciones. Los diagramas de secuencia se utilizan (junto con las tarjetas CRC) para determinar las clases. El diagrama de generalización/especialización (gen/ spec) es un derivado del diagrama de clases.
- Los diagramas de estados, que muestran las transiciones de estado. Cada clase puede crear un diagrama de estados, el cual es útil para determinar los métodos de la clase.



3.3.1. Diagrama de Casos de Uso

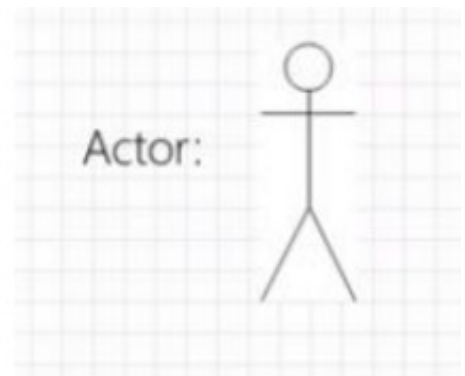
Un modelo de casos de uso muestra una vista del sistema desde la perspectiva del usuario, por lo cual describe qué hace el sistema sin describir cómo lo hace.

- Un caso de uso provee a los desarrolladores un panorama sobre lo que desean los usuarios.
- Está libre de detalles técnicos o de implementación.
- Podemos pensar en un caso de uso como una secuencia de transacciones en un sistema.

- El modelo de casos de uso se basa en las interacciones y relaciones de los casos de uso individuales.

El término actor se refiere a un rol específico de un usuario del sistema. Por ejemplo, un actor puede ser un empleado, pero también puede ser un cliente en la tienda de la empresa. Incluso cuando es la misma persona en el mundo real, se representa como dos símbolos distintos en un diagrama de caso de uso, ya que la persona interactúa con el sistema en distintos roles.

El actor existe fuera del sistema e interactúa con éste de una manera específica. Un actor puede ser un humano, otro sistema o un dispositivo como un teclado o una conexión Web. Los actores pueden iniciar una instancia de un caso de uso. Un actor puede interactuar con uno o más casos de uso; un caso de uso puede involucrar a uno o más actores.



Un caso de uso siempre describe tres cosas: un actor que inicia un evento, el evento que desencadena un caso de uso y el caso de uso que realiza las acciones desencadenado por el evento. En un caso de uso, un actor que utiliza el sistema inicia un evento que comienza una serie relacionada de interacciones en el sistema. Los casos de uso se utilizan para documentar una transacción o evento únicos. Un evento es una entrada para el sistema que ocurre a una hora y lugar específicos, y provoca que el sistema haga algo.

Hay cuatro tipos básicos de relaciones de comportamiento:

comunica, incluye, extiende y generaliza.

Relación	Símbolo	Significado
Comunica	————	Para conectar un actor con un caso de uso se utiliza una línea sin puntas de flecha.
Incluye	<< Incluye >> ←-----	Un caso de uso contiene un comportamiento común para más de un caso de uso. La flecha apunta al caso de uso común.
Extiende	<< Extiende >> ----->	Un caso de uso distinto maneja las excepciones del caso de uso básico. La flecha apunta del caso de uso extendido al básico.
Generaliza	————>	Una "cosa" de UML es más general que otra "cosa". La flecha apunta a la "cosa" general.

Antes de la versión 1.3 de UML se llamaba "use"

COMUNICACIÓN

Conecta actor con caso de uso. La tarea del caso de uso es proporcionar cierto tipo de resultado que sea benéfico para el actor en el sistema.

INCLUSIÓN

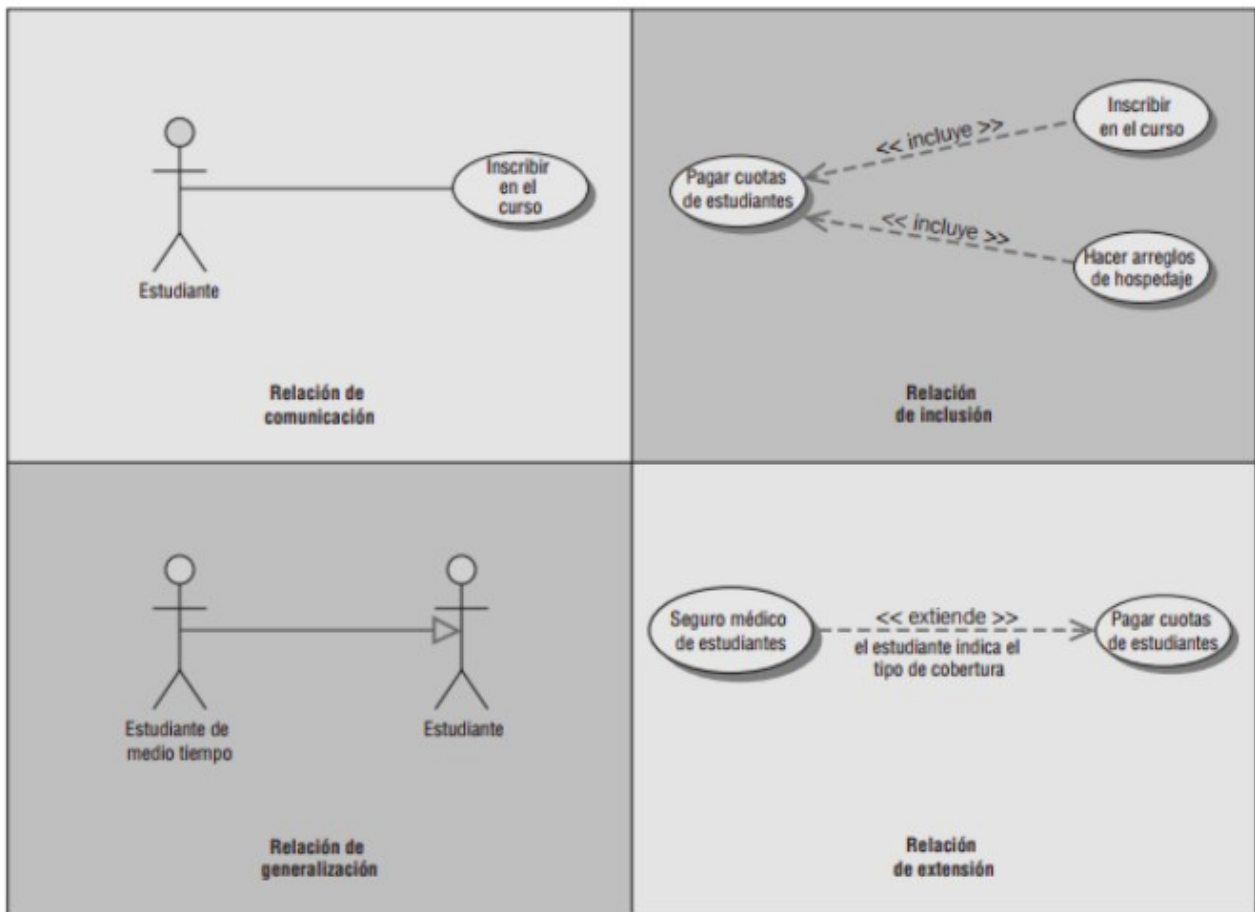
Un caso de uso contiene un comportamiento común para más de un caso de uso. Antiguamente se llamaba usar.

EXTENSIÓN

Un caso de uso posee el comportamiento que permite al nuevo caso de uso manejar una variación o excepción a partir del caso de uso básico.

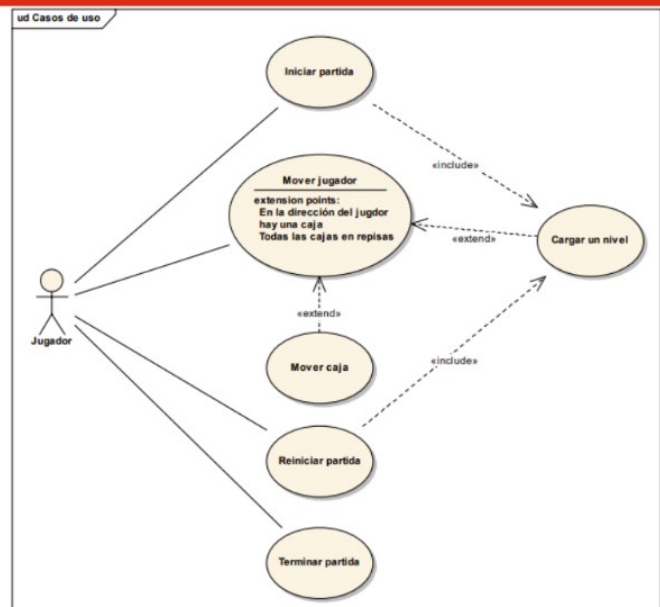
GENERALIZACIÓN

Esta relación implica que una cosa es más común que otra. Puede existir entre dos actores o dos casos de uso.



Casos de uso - Ejercicio

- Sokoban es un juego de varios niveles.
- Cada nivel está compuesto por un jugador, cajas, repisas y muros.
- El objetivo del jugador es empujar todas las cajas sobre las repisas.
- Cuando esto sucede el jugador pasa al siguiente nivel.
- Para mover una caja, el jugador debe colocarse al lado y empujarla.
- Si la casilla hacia la que está empujando la caja está libre la caja se moverá.
- Si el jugador se queda bloqueado, es decir, no puede terminar el nivel, puede reiniciar el nivel perdiendo una vida.
- Cuando el jugador pierde todas sus vidas la partida termina.



Casos de uso – Descripción / Escenario

Caso de Uso	[Nombre del Caso de Uso]	Identificador: [Del caso de uso]
Actores	[Listado de los actores que tienen participación en el caso de uso]	
Tipo	[Tipo de caso de uso, primario, secundario, opcional]	
Referencias	[Requerimientos o funcionalidades incluidas en este caso de uso. Casos de uso relacionados.]	
Precondición	[Condiciones sobre el estado del sistema que deben cumplirse para iniciar el caso de uso]	
Postcondición	[Efectos inmediatos que tienen la ejecución del caso de uso sobre el estado del sistema]	
Descripción	[Descripción del caso de uso]	
Resumen	[Resumen de alto nivel del funcionamiento]	

Nro.	Ejecutor	Paso o Actividad
[Nro. de paso]	[Actor ejecutor o especifica si es el sistema o subsistema]	[Descripción del paso actividad ejecutado]

[Se describe el proceso o secuencia de pasos ejecutados usando frases cortas]
 [Cada paso del proceso puede ser ejecutado por los Actores o por el sistema]
 [Se describe la secuencia de acciones realizadas por los actores y la secuencia de actividades realizada por el sistema como respuesta].

Curso
normal

Nro.	Descripción de acciones alternas
[Número de paso]	[Descripción de la secuencia de acciones alternas para el número de actividad indicado. Debe hacer referencia al número de paso en el curso normal]

Curso
alternativo

[Cada paso descrito en el curso normal, puede tener actividades alternas, según la distribución de escenarios que ocurra en el flujo de procesos, en esta ficha se completa para cada actividad (haciendo referencia a su número) las posibles secuencias alternas]

3.3.2. Diagrama de Actividad

Los diagramas de actividad muestran la secuencia de actividades en un proceso, incluyendo las actividades secuenciales y paralelas, además de las decisiones que se toman. Por lo general se crea un diagrama de actividad para un caso de uso y puede mostrar los distintos escenarios posibles.

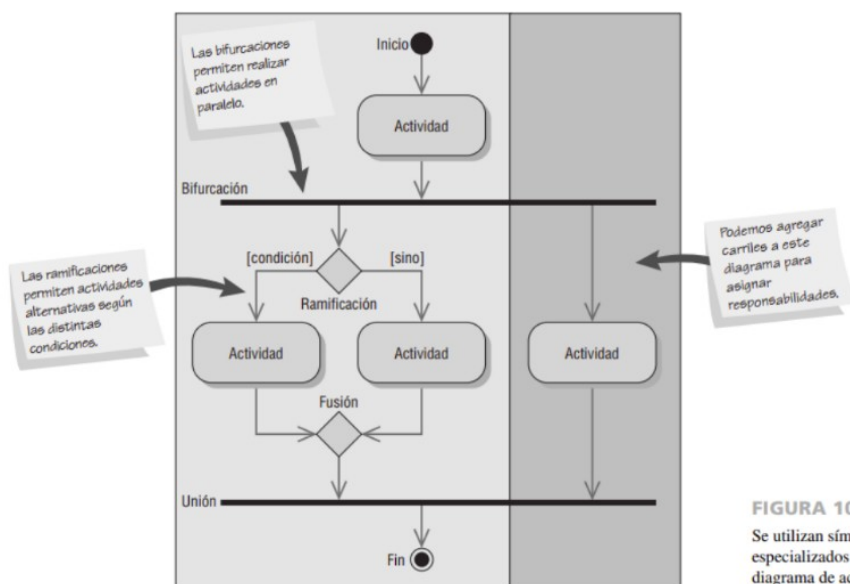


FIGURA 10.8
Se utilizan símbolos especializados para dibujar un diagrama de actividad.

1. Un rectángulo con esquinas redondas representa una actividad, ya sea manual —como firmar un documento— o automatizada — como un método o programa—.
2. Una flecha representa a un evento. Los eventos representan cosas que ocurren en cierto momento y lugar.
3. Un diamante representa una decisión (también conocida como ramificación) o una fusión. En las decisiones hay una flecha que entra al diamante y varias que salen de él. Se puede incluir una condición de guardia, que muestra los valores de la condición. Las fusiones muestran varios eventos que se combinan para formar un evento.
4. Un rectángulo largo y plano representa una barra de sincronización. Estas barras se utilizan para mostrar las actividades paralelas, donde puede haber un evento que entre a la barra de sincronización y varios eventos que salgan de ella, a lo cual se le denomina bifurcación. Una sincronización en la que varios eventos se fusionan en uno solo se denomina unión.
5. El estado inicial se muestra como un círculo relleno. El estado final se muestra como un círculo negro rodeado por un círculo blanco.
6. Los rectángulos que rodean otros círculos se denominan carriles (swimlanes). Estos carriles indican particionamiento y se utilizan para mostrar qué actividades se realizan en cada plataforma; también

muestran las actividades que realizan distintos grupos de usuarios. Los carriles son zonas que pueden describir tanto la lógica como la responsabilidad de una clase.

Creación de diagramas de actividad

Para crear diagramas de actividad hay que preguntarse qué ocurre primero y qué ocurre después.

Debemos determinar si las actividades se realizarán en secuencia o en paralelo. Si se crearon escenarios de casos de uso, hay que examinarlos para determinar la secuencia de las actividades.

Buscar los lugares en donde se toman decisiones y preguntar qué ocurre en cada uno de los resultados de esas decisiones. Cada ruta a través de las diversas decisiones incluidas en el caso de uso es un escenario distinto.

Los carriles son útiles para mostrar cómo se debe transmitir o convertir datos, tales como los que van del sitio Web al servidor, o del servidor a la base de datos.

Se pueden usar diagramas de actividad para construir planes de prueba. Hay que evaluar cada evento para ver si el diagrama de actividad pasa al siguiente estado. Hay que evaluar cada decisión para ver si se toma la ruta correcta cuando ocurren las condiciones de decisión.

Se pueden usar diagramas de actividad para construir planes de prueba. Hay que evaluar cada evento para ver si el diagrama de actividad pasa al siguiente estado. Hay que evaluar cada decisión para ver si se toma la ruta correcta cuando ocurren las condiciones de decisión.

Los diagramas de actividad no se utilizan para todos los casos. Debe usar diagramas de actividad cuando:



El analista no necesita un diagrama de actividad cuando el caso de uso es simple, o cuando hay que modelar el cambio de estado.

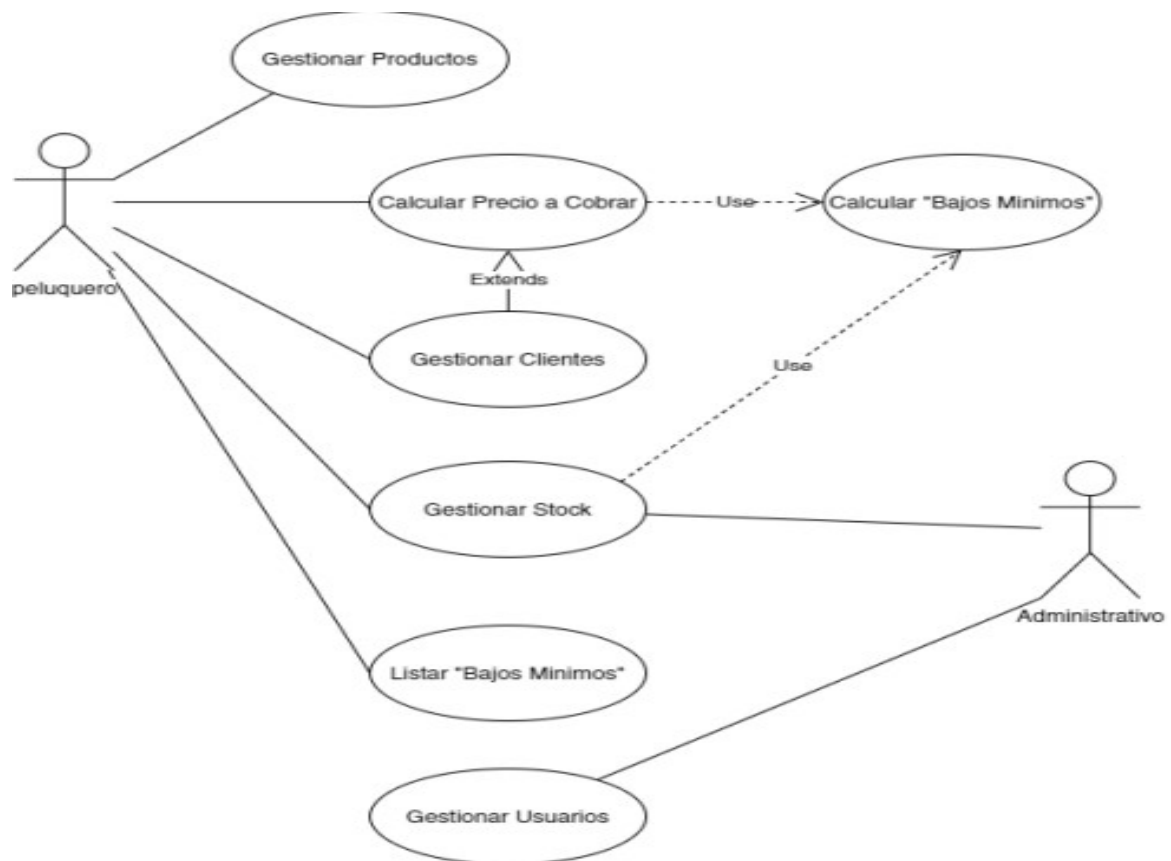
Ejemplo

Se desea desarrollar una aplicación de gestión para una peluquería con la finalidad de llevar la gestión de productos y clientes (EPELUC).

Esta aplicación permitirá la gestión de productos, es decir, altas, bajas y modificaciones de productos, así como actualizar el stock de cada uno de ellos. Por otra parte, y respecto a los clientes, SIPEL permitirá dar de alta, baja y modificar clientes. El sistema también permite calcular el precio de los servicios que ha consumido el cliente en la peluquería (cortar el pelo, lavar el pelo, teñir el pelo, secar, etc.), permitiendo al peluquero/a indicar los productos consumidos (champú, crema suavizante, tinte, laca, etc.). En el momento de calcular dicho coste, el sistema modifica automáticamente la cantidad de los productos consumidos. Finalmente, si coincide que ese día es el cumpleaños del cliente, se le bonifica con un descuento del 20% sobre el precio total y se genera un aviso indicando esta situación.

Habrà que comprobar si un producto está bajo-mínimos (es decir, si el stock del producto es inferior o igual a un stock mínimo admisible) en dos ocasiones: (a) cuando se calcula el precio del cliente, tras la modificación automática de la cantidad de cada producto consumido; (b) cuando se actualiza el stock de los productos. Ambas comprobaciones marcarán aquellos productos que están bajo-mínimos, mostrando un aviso de que hay productos en esta situación.

Otra de las funcionalidades de dicho sistema será permitir al peluquero/a obtener un listado de los productos bajo-mínimos, es decir, de aquellos productos marcados previamente como tal.

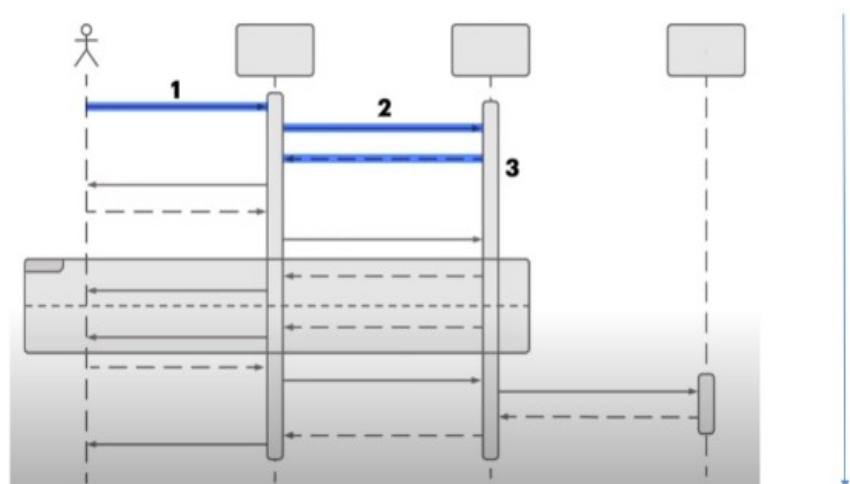


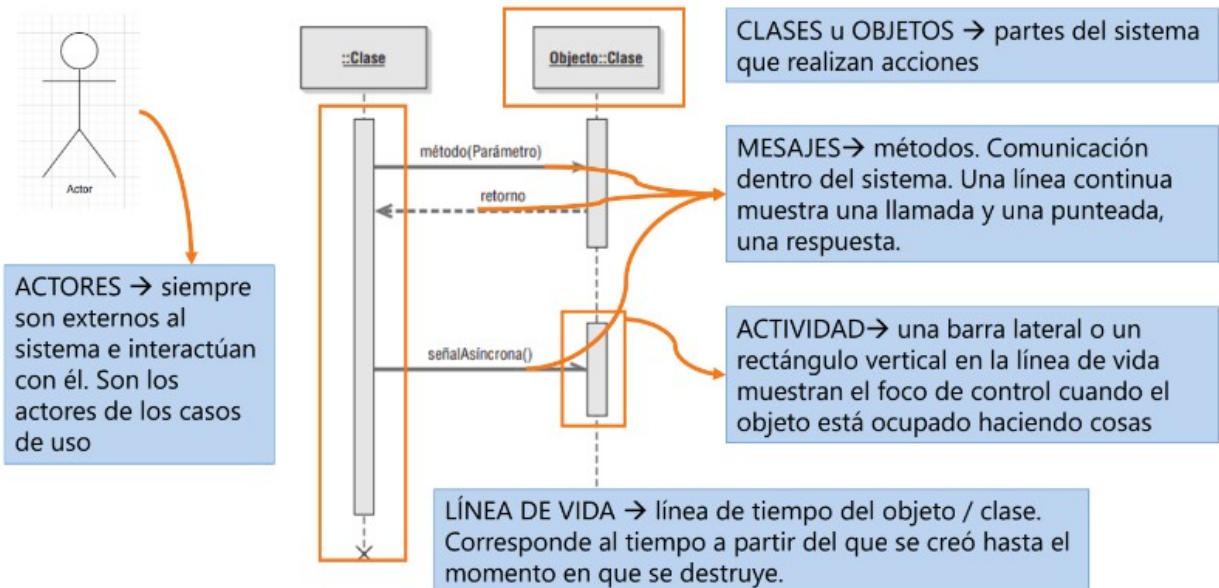
3.3.3. Diagrama de Secuencia

Los diagramas de secuencia pueden ilustrar una sucesión de interacciones entre clases o instancias de objetos a través del tiempo. A menudo, los diagramas de secuencia se utilizan para ilustrar el procesamiento descrito en los escenarios de casos de uso.

En la práctica, los diagramas de secuencia se derivan del análisis de casos de uso y se utilizan en el diseño de sistemas para derivar las interacciones, las relaciones y los métodos de los objetos en el sistema. Los diagramas de secuencia se utilizan para mostrar el patrón general de las actividades o interacciones en un caso de uso. Cada escenario de caso de uso puede crear un diagrama de secuencia, aunque éstos no siempre se crean para escenarios de menor importancia.

En estos diagramas se muestran las interacciones en ORDEN. Se podría decir entonces, que es un diagrama para visualizar la secuencia de los eventos que ocurren en un sistema.





¿Quién inicia la acción? ¿es externo o interno?

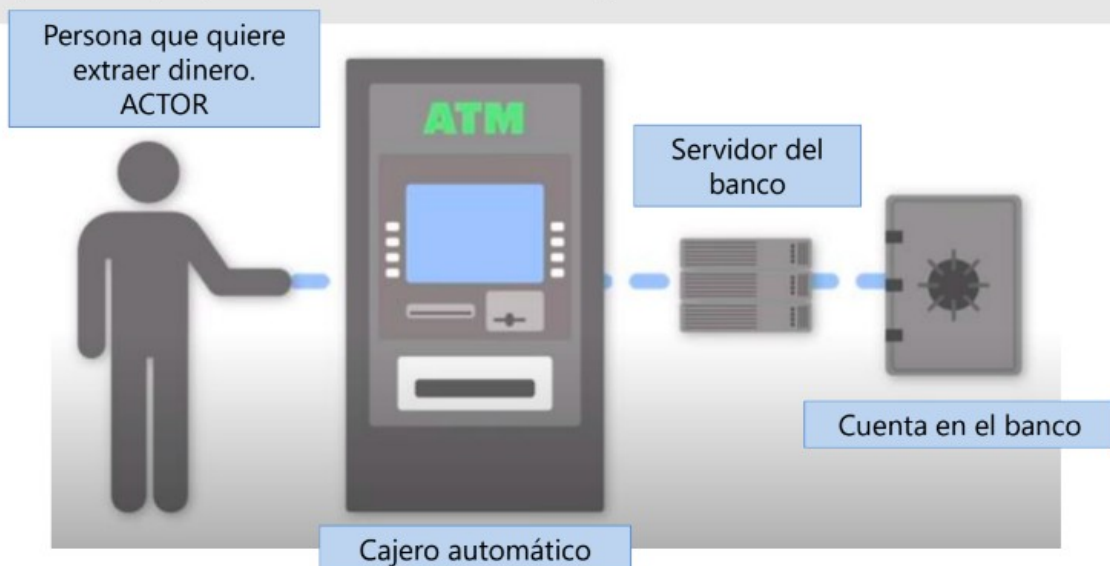
¿Cómo se inicia la interacción?

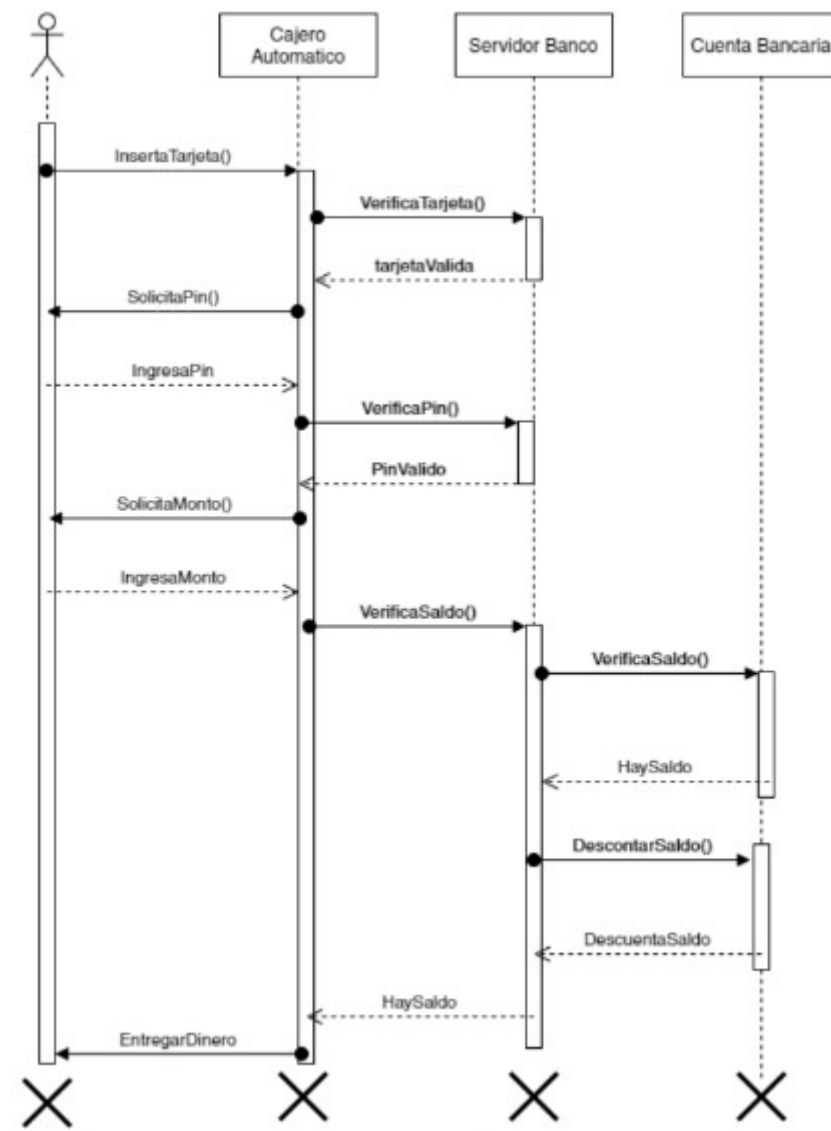
¿Qué parte del sistema interactúa en primer lugar?

¿Qué partes del sistema interactúan?

Ejemplo

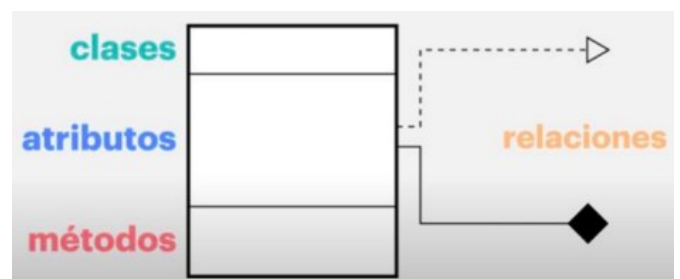
Vamos a poner un ejemplo con una extracción en un cajero automático:





3.3.4. Diagrama de Clases

Las metodologías orientadas a objetos trabajan para descubrir las clases, atributos, métodos y relaciones entre las clases. Como la programación ocurre a nivel de clase, definir clases es una de las tareas más importantes del análisis orientado a objetos. Los diagramas de clases muestran las características estáticas del sistema y no representan ningún procesamiento en especial. Un diagrama de clases también muestra la naturaleza de las relaciones entre las clases.



(campos, variables, propiedades)

ATRIBUTO

- Un dato importante que contiene valores que describen cada instancia de esa clase

(operaciones, funciones)

MÉTODOS

- Permiten especificar cualquier características de comportamiento de una clase

VISIBILIDAD

• Define la accesibilidad para ese atributo o método

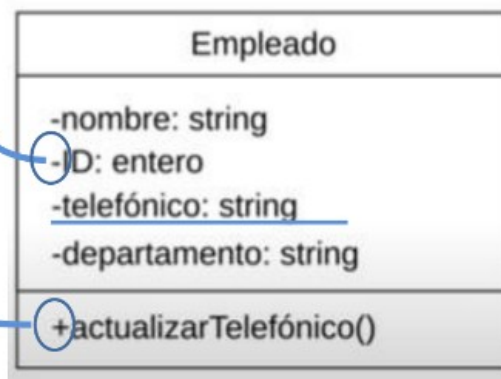
- PRIVADO: ninguna otra clase o subclase puede acceder a ellos

+ PÚBLICO: cualquiera otra clase o subclase puede acceder a ellos

PROTEGIDO: la misma clase o subclase puede acceder a ellos

En general los atributos son privados o protegidos. Se accede al objeto mediante sus métodos.

En cambio, muchos de los métodos son públicos, ya que sirven para interactuar con las clases.



El nombre de la clase comienza con mayúsculas

Los atributos se escriben en minúscula. Luego se puede poner ":" y el tipo de datos que debe ir.

Los métodos comienzan con minúsculas y todas las palabras van sin espacio, comenzando con mayúsculas. Luego () lo cual indica que se pueden pasar datos como parámetros junto con el mensaje

Hacer los atributos privados implica que serán visibles sólo para los objetos externos a través de los métodos de la clase, una técnica que **se conoce como encapsulamiento** u ocultamiento de la información.

Tipos de Clases

CLASES DE ENTIDAD

- Las clases de entidad representan elementos del mundo real como personas o cosas.
- El analista necesita determinar qué atributos debe incluir en las clases. Cada objeto tiene muchos atributos, pero la clase debe incluir sólo aquellos que la organización utilice.

CLASES DE LÍMITE O DE INTERFAZ

- Las clases de límite o de interfaz proveen los medios para que los usuarios trabajen con el sistema. Hay dos amplias categorías de clases de interfaz: humana y de sistema. Una interfaz humana puede ser una pantalla, una ventana, un formulario Web, etc.
- Las interfaces del sistema necesitan enviar o recibir datos de otros sistemas.

CLASES ABSTRACTAS

- Las clases abstractas son clases que no se pueden instanciar en forma directa. Las clases abstractas están enlazadas a clases concretas en una relación de generalización / especialización (gen/spec).

CLASES DE CONTROL

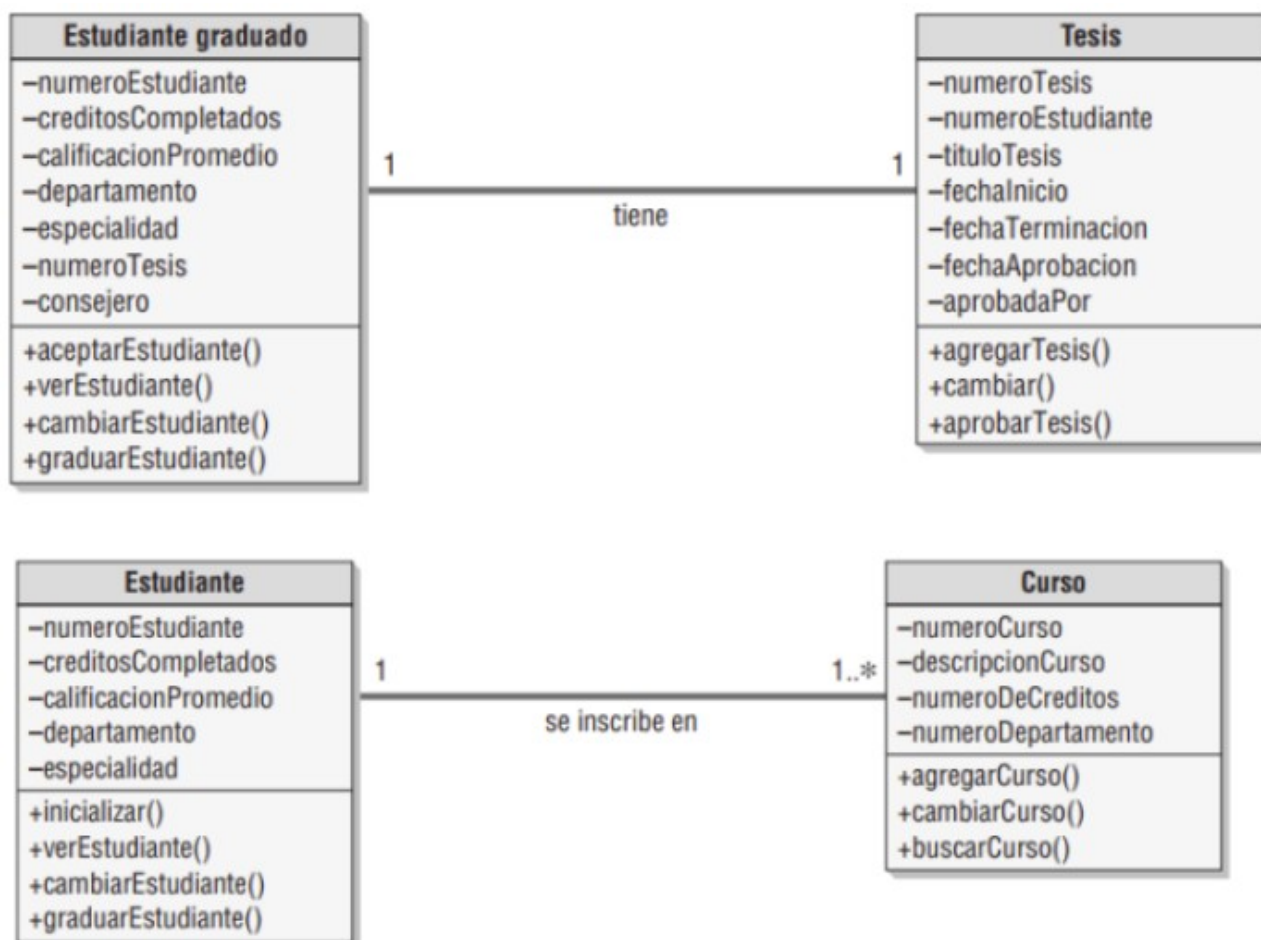
- Las clases de control o activas se utilizan para controlar el flujo de actividades; actúan como un coordinador a la hora de implementar las clases. Para obtener clases que se puedan reutilizar, un diagrama de clases puede incluir muchas clases de control pequeñas. A menudo las clases de control se derivan durante el diseño del sistema

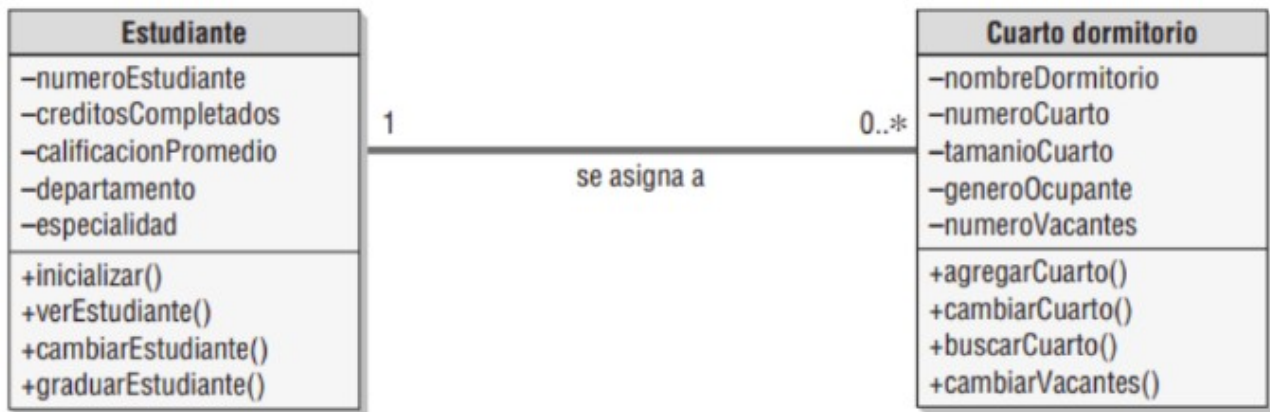
Las reglas para crear diagramas de secuencia son que todas las clases de interfaz deben estar conectadas a una clase de control. De manera similar, todas las clases de entidad deben estar conectadas a una clase de control. Las clases de interfaz, a diferencia de las otras dos, nunca se conectan de manera directa a las clases de entidad.

Las relaciones son conexiones entre las clases. Se muestran como líneas que conectan a las clases en un diagrama de clases. Hay dos categorías de relaciones: asociaciones y relaciones entre un todo y sus partes.

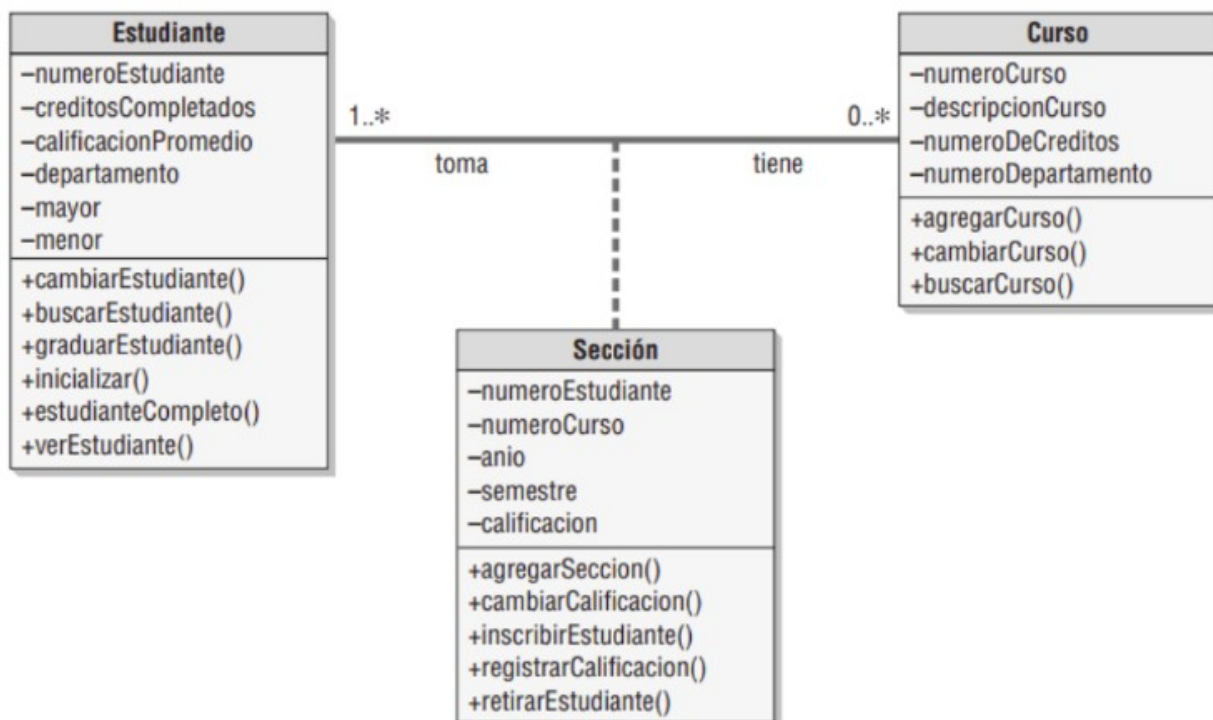
Relaciones Tipo	Relaciones Subtipo	Elemento de UML	Descripción
Asociaciones			Conexión estructural entre clases
Todo/ Parte	Agregación		Existencia independiente de las partes (Asociación débil)
	Composición		Existencia dependiente de las partes (Asociación fuerte)
Generalización			Indica que una clase es una cosa específica de la clase más general

Las asociaciones se muestran como una simple línea en un diagrama de clases. Los puntos finales de la línea se etiquetan con un símbolo que indica la multiplicidad. Un cero representa ninguno, un uno representa uno y sólo uno; un asterisco representa muchos. La notación 0..1 representa de cero a uno y la notación 1..* representa de uno a muchos.





Las clases de asociaciones son aquellas que se utilizan para descomponer una asociación de muchos a muchos entre las clases. Por ejemplo: Estudiante y Curso tienen una relación de muchos a muchos; para resolver esta relación se agrega una clase de asociación llamada Sección entre las clases de Estudiante y Curso conectada con una línea punteada.



Un objeto en una clase puede tener una relación con otros objetos en la misma clase; a esto se le denomina asociación reflexiva. Un ejemplo sería una tarea que tenga una tarea precedente, o un empleado que supervisa a otro empleado. Esto se muestra como una línea de asociación que conecta a la clase consigo misma, con etiquetas que indican los nombres de los roles, como tarea y tarea precedente.

Relaciones Entre Un Todo Y Sus Partes

Estas relaciones se dan cuando una clase representa al objeto como un todo y otras clases representan partes de ese objeto. El todo actúa como un contenedor para las partes. Para mostrar estas relaciones en un diagrama de clases se utiliza una línea con un diamante en un extremo. El diamante está conectado al objeto que es el todo.

Una relación entre un todo y sus partes puede ser un objeto entidad que tenga partes distintas, como un sistema computarizado que incluye computadora, impresora, pantalla o describir una interfaz de usuario en la que una pantalla de GUI contiene una serie de objetos como listas, cuadros o botones de opción, o tal vez un encabezado, cuerpo y área del pie. Las relaciones entre un todo y sus partes tienen tres categorías: agregación, colección y composición

Agregación

- La agregación provee el medio para mostrar que todo el objeto está compuesto de la suma de sus partes (otros objetos). En el ejemplo de inscripción de estudiantes, el departamento *tiene un* curso y el curso *es para un* departamento. Ésta es una relación más débil, ya que tal vez el departamento cambie o se elimine y el curso puede seguir existiendo.

Colección

- Una colección consiste de un todo y sus miembros. Éste puede ser un distrito de votación con votantes o una biblioteca con libros. Los votantes o libros pueden cambiar, pero el todo retiene su identidad. Ésta es una asociación débil.

Composición

- La composición es una relación entre un todo y sus partes, donde el todo tiene una responsabilidad por cada parte. Es una relación más fuerte y por lo general se muestra con un diamante relleno. Las palabras clave para la composición son una clase "siempre contiene" a otra clase. Si el todo se elimina, todas las partes se eliminan.

Generalización/Especialización

HERENCIA

- Varias clases pueden tener los mismos atributos y/o métodos. Cuando esto ocurre, se crea una clase general que contiene los atributos y métodos comunes. La clase especializada recibe o hereda los atributos y métodos de la clase general. Además, la clase especializada tiene atributos y métodos únicos que sólo se definen en ella. Al crear clases generalizadas y permitir que la clase especializada herede los atributos y métodos ayudamos a fomentar la reutilización, ya que el código se utiliza muchas veces. Esto también ayuda a mantener el código existente del programa. Así, el analista puede definir los atributos y métodos una vez, pero usarlos muchas veces en cada clase heredada.

POLIMORFISMO

- El polimorfismo (que significa muchas formas) o redefinición de métodos (no es lo mismo que sobrecarga de métodos) es la capacidad de un programa orientado a objetos de tener varias versiones de un mismo método con el mismo nombre dentro de una relación superclase/subclase. La subclase hereda un método padre pero le puede agregar elementos o modificarlo. La subclase puede cambiar el tipo de los datos, o puede cambiar la forma en que trabaja el método. Cuando los atributos o métodos se definen más de una vez, se utiliza el más específico (el más bajo en la jerarquía de clases). El programa compilado recorre la cadena de clases en busca de los métodos.

CLASES ABSTRACTAS

- Las clases abstractas son clases generales y se utilizan cuando se incluye la generalización/especificación (gen/spec) en el diseño. La clase general se convierte en la clase abstracta. La clase abstracta no tiene objetos directos o instancias de clase, y se utiliza sólo en conjunto con clases especializadas. Por lo general las clases abstractas tienen atributos y pueden tener unos cuantos métodos.
- Por ejemplo, un auto *es un* vehículo y un camión *es un* vehículo. En este caso, el vehículo es una clase abstracta creada para tener atributos y métodos en común, mientras que auto y camión son las cosas más específicas

Pasos para diagrama de clases

Busque los sustantivos en los casos de uso y haga una lista. Son objetos potenciales. Una vez que identifique los objetos, busque similitudes y diferencias en los objetos debido a sus estados o comportamiento y después cree las clases.

1

Examine los diagramas de casos de uso y de secuencias para poder determinar las clases.

2

Defina las principales relaciones entre las clases. Busque las relaciones "tiene un" y "es un" entre las clases.

3

Empezando con los casos de uso que son los más importantes para el diseño del sistema, cree los diagramas de clases que muestren las clases y relaciones que existen en los casos de uso. Un diagrama de clases puede representar a las clases y relaciones descritas en varios casos de uso relacionados.

4